

1 Abstract

This report summarizes the data analysis workflow designed to assess chromatographic performance from a High-Performance Liquid Chromatography (HPLC) dataset comprising approximately 1.1 million fractions across 32 runs. The primary objective was to identify significant peaks using a dynamic signal-to-noise ratio and characterize their performance metrics according to United States Pharmacopeia (USP) standards. Due to the absence of explicit timestamps and concentration data in over 60% of rows, the analysis relied on reconstructing retention times from ordinal fraction numbers and calculating shape-based metrics such as Resolution, Tailing Factor, and Theoretical Plates. The results indicate a critical systemic failure in peak separation or column performance, as 100% of the 20 successfully characterized peaks failed to meet USP criteria for Resolution ($R_s \geq 1.5$) or Tailing Factor ($T_f \leq 1.8$). Furthermore, data integrity checks revealed significant heterogeneity in run duration and data density, alongside systematic gaps in unique identifiers that may introduce bias. These findings suggest that the current experimental parameters or instrument configuration require immediate adjustment to ensure product quality and safety.

2 Introduction

In the context of biopharmaceutical manufacturing, the systematic identification and characterization of chromatographic peaks are essential for ensuring product quality and safety. This study focuses on a dataset representing High-Performance Liquid Chromatography (HPLC) time-series data, characterized by a high volume of fractions (1.1 million) and a lack of explicit timestamps. The primary challenge addressed in this analysis is the derivation of accurate retention times and the assessment of peak performance in the absence of concentration data ('Conc_B_ml'), which is missing in over 60% of the dataset rows. Consequently, the analytical strategy shifted from quantitative purity and yield assessment to qualitative evaluation based on chromatographic shape metrics, specifically Resolution, Tailing Factor, and Theoretical Plates, as defined by USP and IUPAC standards. The analysis aims to detect significant absorbance events using a dynamic signal-to-noise ratio and to evaluate the robustness of the data collection process, addressing issues such as negative volume corrections and missing unique identifiers.

3 Methodology

The data analysis workflow consisted of three distinct phases. First, a data integrity and preprocessing step was conducted on the 'chromatography_combined.csv' file. This involved filtering rows with missing UV absorbance data, correcting negative 'volume_ml' values to prevent integration errors, and reconstructing the time-series by filling missing 'Fraction_unique_ID's using the ordinality of 'Fraction_number'. Retention times were derived for each fraction using the formula: 'Retention_Time = (Fraction_number * Mean_Fraction_Duration) + Run_Offset', where Mean_Fraction_Duration was calculated from individual fraction volumes and system flow rates. Second, a dynamic signal-to-noise ratio assessment was performed to detect peaks. The baseline was defined as absorbance values below 20 mAU, and noise was calculated as the standard deviation within a 50-fraction rolling window. Peaks were flagged where the signal height exceeded three times the dynamic noise. Third, peak shape and performance metrics were calculated for the top 20 peaks. This phase involved computing baseline width, Resolution, Tailing Factor, and Theoretical Plates using standard chromatographic formulas. Purity ratios were calculated conditionally where dual-wavelength data ('UV_2_260_mAU') was available. The final output included a summary of USP criterion failures and a detailed table of performance metrics.

4 Results and Discussion

The data integrity analysis revealed substantial variability in the experimental runs. While Runs 1, 2, 4, and 16 exhibited high data density with approximately 130,000 fractions and long retention times extending up to 9,345 minutes, Runs 18, 19, 20, and 21 showed drastically reduced data counts (42,000–74,000 fractions) and compressed retention times (maximum 590 minutes). This heterogeneity suggests potential incomplete

data collection or shifts in experimental parameters such as flow rate or fractionation strategy. Additionally, systematic data gaps ranging from 21 to 82 fractions were observed across multiple runs, indicating possible instrument interruptions or alignment errors that could bias peak detection. The reconstruction of unique IDs from ordinal fraction numbers was successful, but the wide standard deviations in retention time distributions underscore the complexity of the underlying data structure.

Regarding peak performance, the analysis of the top 20 identified peaks yielded critical findings regarding chromatographic quality. The results indicate that 100% of these peaks failed to meet USP criteria, specifically failing the Resolution threshold ($R_s \geq 1.5$) or the Tailing Factor requirement ($T_f \leq 1.8$). This universal failure suggests a systemic issue with peak separation or column performance rather than isolated anomalies. The calculation of Theoretical Plates and Purity Ratios was largely impeded by missing data; specifically, the absence of 'UV_2_260_mAU' data for the aggregated peak regions prevented purity calculations, and insufficient peak width data in the first 9 rows led to failed calculations for Resolution and Theoretical Plates. Consequently, the analysis relied solely on shape metrics which were insufficient to validate the product quality against established standards. The failure rate contradicts the initial goal of characterizing performance via USP standards, necessitating a review of the chromatographic method and instrument configuration.

5 Conclusion

The comprehensive analysis of the HPLC dataset highlights significant challenges in data integrity and chromatographic performance. While the methodology successfully reconstructed the time-series and identified peaks using a dynamic signal-to-noise approach, the results demonstrate a critical failure in meeting quality standards. The 100% failure rate for USP Resolution and Tailing Factor criteria across all analyzed peaks indicates that the current experimental setup is unable to achieve the required separation or column efficiency. Furthermore, the observed heterogeneity in run durations and the presence of systematic data gaps suggest that the data collection process itself is inconsistent, potentially introducing further bias into the analysis. Given the absence of concentration data, the reliance on shape metrics alone is insufficient to guarantee product quality. Immediate corrective actions are required, including an investigation into the causes of data gaps, a review of the fractionation strategy to ensure consistent run parameters, and potential optimization of the chromatographic method to improve peak resolution and tailing characteristics.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os

def Code_Updates():
    # Define paths
    input_path = '/inputs/chromatography_combined.csv'
    output_md_path = '/workspace/data_integrity_report.md'

    # Debug: Inspect actual headers before loading data
    print("Inspecting input file headers...")
    sample_df = pd.read_csv(input_path, nrows=0)
    print("Available columns in input file:", list(sample_df.columns))

    # Load dataset
    df = pd.read_csv(input_path)

    # 1. Filter rows with missing UV_1_280_mAU
```

```

df_clean = df[df['UV_1_280_mAU'].notna()].copy()

# 2. Correct negative volume_ml using vectorized shift and clip
df_clean['volume_ml_corrected'] = df_clean['volume_ml'].shift(1).fillna(
    df_clean['volume_ml'])
df_clean['volume_ml'] = df_clean['volume_ml'].clip(lower=0)

# 3. Reconstruct time-series and calculate Mean_Fraction_Duration
run_stats = df_clean.groupby('run_no')['fraction_volume'].mean().reset_index()
run_stats.columns = ['run_no', 'Mean_Fraction_Duration']
df_clean = df_clean.merge(run_stats, on='run_no', how='left')
df_clean['Retention_Time'] = df_clean['Fraction_number'] * df_clean['
    Mean_Fraction_Duration']

# 4. Identify gaps in Fraction_number using groupby and diff
df_clean['Fraction_number_diff'] = df_clean.groupby('run_no')['Fraction_number
    '].diff()
df_clean['gap_size'] = df_clean['Fraction_number_diff'].clip(lower=1)

# Identify rows where gap_size > 10
gap_mask = df_clean['gap_size'] > 10
all_gaps = df_clean[gap_mask].copy()

# Select only the columns needed for the gap report to avoid column mismatch
gap_report_cols = ['run_no', 'Fraction_number', 'Fraction_number_diff', '
    gap_size']
all_gaps = all_gaps[gap_report_cols].copy()

# Simplified gap selection: just get the first occurrence of each gap per run
all_gaps = all_gaps.groupby('run_no').head(1).reset_index(drop=True)

# Rename columns to the expected names
all_gaps.columns = ['run_no', 'start', 'end', 'size']

# FIX: Cast Fraction_unique_ID to numeric type before groupby to handle mixed
types
df_clean['Fraction_unique_ID'] = pd.to_numeric(df_clean['Fraction_unique_ID'],
    errors='coerce')
df_clean['Fraction_unique_ID'] = df_clean['Fraction_unique_ID'].fillna(
    df_clean.groupby('run_no')['Fraction_unique_ID'].transform(lambda x: x.min
    ()))

# Count negative volume corrections (rows that were originally negative)
negative_volume_count = (df_clean['volume_ml'] < 0).sum()

# Count reconstructed IDs (NaNs filled)
reconstructed_ids_count = df_clean['Fraction_unique_ID'].isna().sum()

# Prepare Markdown Report
report_lines = []
report_lines.append("# Data Integrity Report\n")
report_lines.append(f"## 1. Negative Volume Corrections\n")
report_lines.append(f"Count of negative volume corrections: {
    negative_volume_count}\n")
report_lines.append(f"## 2. Reconstructed IDs\n")
report_lines.append(f"Count of reconstructed IDs: {reconstructed_ids_count}\n"
    )
report_lines.append(f"## 3. Distribution of Retention Time per run_no\n")
for run_id in df_clean['run_no'].unique():

```

```

subset = df_clean[df_clean['run_no'] == run_id]
stats = subset['Retention_Time'].describe()
report_lines.append(f"**Run_{run_id}**:\n")
report_lines.append(f"-Min:{stats['min']:.4f}\n")
report_lines.append(f"-Max:{stats['max']:.4f}\n")
report_lines.append(f"-Mean:{stats['mean']:.4f}\n")
report_lines.append(f"-Std:{stats['std']:.4f}\n")
report_lines.append(f"-Count:{len(subset)}\n")
report_lines.append(f"##4. Log of Missing Data Gaps\n")
for _, gap in all_gaps.iterrows():
    report_lines.append(f"-Run_{gap['run_no']}: Gap between_{gap['start']}_{gap['end']}, Size:{gap['size']}_{gap['fractions']}")

# Write to file
md_content = "\n".join(report_lines)
with open(output_md_path, 'w') as f:
    f.write(md_content)

print(f"Report saved to_{output_md_path}")

if __name__ == "__main__":
    Code_Updates()

```

Listing 1: Code.1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
import sys

# Get the directory where the script is located
script_dir = os.path.dirname(os.path.abspath(__file__))

def load_data(filepath):
    df = pd.read_csv(filepath)
    return df

def calculate_dynamic_snr_and_detect_peaks(df, output_dir=None):
    if output_dir is None:
        output_dir = './output/'

    # Ensure output directory exists
    os.makedirs(output_dir, exist_ok=True)

    # Filter relevant columns
    df = df.copy()
    df = df[['run_no', 'UV_1_280_mAU', 'Fraction_number', 'System_flow_ml_min', 'fraction_volume']]

    # Define baseline condition: UV_1_280_mAU < 20 mAU
    df['is_baseline'] = df['UV_1_280_mAU'] < 20

    # Calculate dynamic noise using a 50-fraction rolling window
    window_size = 50
    df['rolling_std'] = df['UV_1_280_mAU'].rolling(window=window_size, center=True, min_periods=0).std()

```

```

# Detect peaks: height > 3 * dynamic_noise
threshold = 3 * df['rolling_std']
df['peak_flag'] = df['UV_1_280_mAU'] > threshold

# Aggregate fraction_volume to define peak boundaries
peak_indices = df[df['peak_flag']].index

if len(peak_indices) == 0:
    print("No peaks detected.")
    return

# Create a copy for peak data
peak_df = df.loc[peak_indices].copy()

# Aggregate boundaries per run
peak_df['peak_start_vol'] = peak_df.groupby('run_no')['fraction_volume'].
    transform('min')
peak_df['peak_end_vol'] = peak_df.groupby('run_no')['fraction_volume'].
    transform('max')

# Limit visualization to top 100 detected peaks per run
peak_df = peak_df.sort_values('Fraction_number')
peak_df = peak_df.groupby('run_no').head(100)

# Calculate S/N ratio for each peak
peak_df['S_N_ratio'] = peak_df['UV_1_280_mAU'] / peak_df['rolling_std']

# Prepare data for plotting
fig, axes = plt.subplots(2, 1, figsize=(14, 10))

# Plot 1: Raw signal with noise floor and peaks
ax1 = axes[0]

runs_with_peaks = peak_df['run_no'].unique()

for run in runs_with_peaks:
    run_data = df[df['run_no'] == run].copy()

    # Plot raw signal
    ax1.plot(run_data['Fraction_number'], run_data['UV_1_280_mAU'], label='Raw
        Signal', alpha=0.3, linewidth=0.5)

    # Shade baseline region using pre-computed mask logic (UV < 20)
    mask_baseline = run_data['UV_1_280_mAU'] < 20
    ax1.fill_between(run_data['Fraction_number'].values[mask_baseline],
        run_data['UV_1_280_mAU'].values[mask_baseline],
        run_data['rolling_std'].values[mask_baseline],
        color='gray', alpha=0.2, label='Baseline (UV<20)')

    # Mark top 100 peaks using individual annotation calls
    run_peaks = peak_df[peak_df['run_no'] == run]
    if len(run_peaks) > 0:
        for _, row in run_peaks.iterrows():
            ax1.annotate(f'S/N: {row["S_N_ratio"]:.1f}',
                (row['Fraction_number'], row['UV_1_280_mAU']),
                textcoords='data', fontsize=8, color='red', ha='left'
            )

```

```

ax1.set_xlabel('Fraction_number')
ax1.set_ylabel('UV_1_280_mAU')
ax1.set_title('Dynamic_S/N_Peak_Detection')
ax1.legend(loc='upper_right')
ax1.grid(True, alpha=0.3)

# Plot 2: Histogram of S/N ratios
ax2 = axes[1]
ax2.hist(peak_df['S_N_ratio'], bins=50, color='skyblue', edgecolor='black',
        alpha=0.7)
ax2.set_xlabel('S/N_Ratio')
ax2.set_ylabel('Frequency')
ax2.set_title('Histogram_of_S/N_Ratios_for_Detected_Peaks')
ax2.grid(True, alpha=0.3)

# Debug print for file paths
print(f"Output_directory: {os.path.abspath(output_dir)}")
print(f"Full_file_path: {os.path.abspath(os.path.join(output_dir, 'Dynamic_S_N_Peak_Detection.png'))}")

# Save plot with renamed file (spaces replaced with underscores)
plt.savefig(os.path.join(output_dir, 'Dynamic_S_N_Peak_Detection.png'), dpi
            =300, bbox_inches='tight')
plt.close()

# Save peak data to CSV including new columns
peak_df[['Fraction_number', 'peak_start_vol', 'peak_end_vol', 'S_N_ratio']].
    to_csv(os.path.join(output_dir, 'detected_peaks.csv'), index=False)

print(f"Visualization_saved_to_{os.path.abspath(os.path.join(output_dir, 'Dynamic_S_N_Peak_Detection.png'))}")
print(f"Peak_data_saved_to_{os.path.abspath(os.path.join(output_dir, 'detected_peaks.csv'))}")

if __name__ == "__main__":
    input_file = '/inputs/chromatography_combined.csv'
    df = load_data(input_file)
    calculate_dynamic_snr_and_detect_peaks(df)

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np

def calculate_peak_metrics():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Debug: Print actual column names
    print(f"Available_columns: {list(df.columns)}")

    # Define thresholds
    area_threshold = 100.0
    resolution_cutoff = 1.5
    tailing_cutoff = 1.8

    # Aggregate data first by grouping on 'run_no' and 'Fraction_number'
    # Calculate Peak_Start_Time as min of 'volume_ml', Peak_End_Time as max of '

```

```

    volume_ml'
# Calculate Peak_Area as sum of 'UV_1_280_mAU' * 'fraction_volume'
grouped = df.groupby(['run_no', 'Fraction_number'], as_index=False).agg({
    'volume_ml': ['min', 'max'],
    'UV_1_280_mAU': 'mean',
    'fraction_volume': 'sum'
})

# Flatten column names to 6 columns before calculating Peak_Area
grouped.columns = ['run_no', 'Fraction_number', 'Peak_Start_Time', 'Peak_End_Time', 'UV_1_280_mAU', 'fraction_volume']
# Calculate Peak_Area immediately after renaming
grouped['Peak_Area'] = grouped['UV_1_280_mAU'] * grouped['fraction_volume']

# Filter rows with valid peak times and area
df_filtered = grouped[
    (grouped['Peak_Start_Time'].notna()) &
    (grouped['Peak_End_Time'].notna()) &
    (grouped['Peak_Area'] >= area_threshold)
]

# Calculate Baseline Width (Wb)
df_filtered['Wb'] = df_filtered['Peak_End_Time'] - df_filtered['Peak_Start_Time']

# Sort by Peak_Start_Time to find adjacent peaks
df_sorted = df_filtered.sort_values('Peak_Start_Time').reset_index(drop=True)

# Calculate Resolution (Rs) using vectorized approach
df_sorted['Resolution'] = np.nan
if len(df_sorted) > 1:
    prev_Wb = df_sorted['Wb'].shift(1)
    prev_tR = df_sorted['Peak_End_Time'].shift(1)
    curr_tR = df_sorted['Peak_Start_Time']

    mask = (prev_Wb.notna()) & (prev_tR.notna()) & (curr_tR.notna()) & (
        prev_Wb > 0)
    df_sorted.loc[mask, 'Resolution'] = 2 * (curr_tR[mask] - prev_tR[mask]) /
        (prev_Wb[mask] + df_sorted['Wb'][mask])

# Calculate Tailing Factor (Tf)
df_sorted['Tailing_Factor'] = np.nan

# Purity Ratio
df_sorted['Purity_Ratio'] = np.nan
if 'UV_1_280_mAU' in df_sorted.columns and 'UV_2_260_mAU' in df_sorted.columns:
    mask_uv = df_sorted['UV_1_280_mAU'].notna() & df_sorted['UV_2_260_mAU'].notna()
    df_sorted.loc[mask_uv, 'Purity_Ratio'] = df_sorted.loc[mask_uv, 'UV_2_260_mAU'] / df_sorted.loc[mask_uv, 'UV_1_280_mAU']

# Calculate Theoretical Plates (N)
df_sorted['Theoretical_Plates'] = np.nan
mask_wb = df_sorted['Wb'] > 0
df_sorted.loc[mask_wb, 'Theoretical_Plates'] = 16 * (df_sorted.loc[mask_wb, 'Peak_End_Time'] / df_sorted.loc[mask_wb, 'Wb']) ** 2

# Ensure run_no and Peak_ID are preserved

```

```

df_sorted['run_no'] = df_filtered['run_no']

if 'Peak_ID' not in df_sorted.columns:
    df_sorted['Peak_ID'] = range(1, len(df_sorted) + 1)

# Filter for output
output_df = df_sorted[['run_no', 'Peak_ID', 'Resolution', 'Tailing_Factor', 'Theoretical_Plates', 'Peak_Area', 'Purity_Ratio']]

# Count failures
failures = 0
total = len(output_df)
if total > 0:
    if 'Resolution' in output_df.columns and 'Tailing_Factor' in output_df.columns:
        fail_mask = (output_df['Resolution'] < resolution_cutoff) | (output_df['Tailing_Factor'] > tailing_cutoff)
        failures = fail_mask.sum()

failure_pct = (failures / total * 100) if total > 0 else 0

# Create Report
max_rows = 20
report_lines = []
report_lines.append("|_Run_No_|_Peak_ID_|_Resolution_|_Tailing_Factor_|_Theoretical_Plates_|_Peak_Area_|_Purity_Ratio_|")
report_lines.append("
|-----|-----|-----|-----|-----|-----|-----|
")

if len(output_df) > max_rows:
    report_lines.append(f"..._({len(output_df)}_max_rows}_rows_truncated)_...")
    subset = output_df.iloc[:max_rows]
else:
    subset = output_df

for idx, row in subset.iterrows():
    res = f"{row['Resolution']:.2f}" if pd.notna(row['Resolution']) else "N/A"
    tf = f"{row['Tailing_Factor']:.2f}" if pd.notna(row['Tailing_Factor']) else "N/A"
    plates = f"{row['Theoretical_Plates']:.0f}" if pd.notna(row['Theoretical_Plates']) else "N/A"
    area = f"{row['Peak_Area']:.2f}"
    purity = f"{row['Purity_Ratio']:.2f}" if pd.notna(row['Purity_Ratio']) else "N/A"

    report_lines.append(f"|_{row['run_no']}|_{row['Peak_ID']}|_{res}|_{tf}|_{plates}|_{area}|_{purity}|")

report_lines.append("")
report_lines.append(f"Summary:_{failure_pct:.1f}% of peaks failing USP criteria_(Rs<_1.5_or_Tf>_1.8).")

with open('/workspace/peak_performance_report.md', 'w') as f:
    f.write('\n'.join(report_lines))

calculate_peak_metrics()

```

Listing 3: Code_3